

OTS06: Lecture Summary

Table of Contents

1. [Overview and Objectives](#)
 2. [Key Steps in Options Backtesting](#)
 3. [Data Collection and Processing](#)
 4. [Data Structure and Cleaning](#)
 5. [Expiry Classification System](#)
 6. [Contract Filtering and Selection](#)
 7. [Event-Based Backtesting Implementation](#)
 8. [Strategy Implementation Examples](#)
 9. [Performance Metrics and Analytics](#)
 10. [Risk Management and Optimisation](#)
-

Overview and Objectives

What is Backtesting?

Backtesting is a systematic method to evaluate trading strategies using historical data by:

- Simulating real-world trading conditions
- Validating strategy performance before live trading
- Measuring key metrics: total return, Sharpe ratio, drawdown
- Identifying potential overfitting and data limitations

Core Objectives

- Build a robust options backtesting framework
 - Implement event-based simulation for realistic execution
 - Create modular, reusable strategy components
 - Generate comprehensive performance analytics
-

Key Steps in Options Backtesting

1. Data Collection and Processing

- Collect options and underlying asset data from sources like optionsDX
- Handle compressed data formats (.7z archives)
- Merge multiple time periods into unified datasets

2. Data Structure and Cleaning

- Standardize column names and formats
- Convert data types (strings to floats/dates)
- Remove unnecessary whitespace and formatting issues

3. Expiry Classification

- Categorize options by expiry type (LEAPS, Monthly, Weekly, Daily)
- Identify key trading days (first/last of month, day before expiry)

4. Contract Filtering

- Filter based on liquidity criteria
- Remove contracts with zero bids/asks
- Apply strike distance and implied volatility filters

5. Event-Based Simulation

- Implement entry/exit conditions
- Track positions and mark-to-market values
- Handle trading costs and slippages

6. Performance Analysis

- Calculate trade-level and strategy-level metrics
 - Generate risk-adjusted return measures
 - Create visualizations and reports
-

Data Collection and Processing

Source and Format

- Data Provider: optionsDX for SPX historical options data
- Format: Compressed .7z archives containing CSV files
- Frequency: Daily option chain data
- Coverage: Multiple expiries, strikes, and Greeks

Implementation Process

python

Key libraries and the extraction process

```
import py7zr
```

```
import pandas as pd
```

Extract .7z files

```
with py7zr.SevenZipFile(zip_path, mode='r') as archive:
```

```
    archive.extractall(extract_path)
```

Merge monthly data files

```
dfs = []
```

```
for file in csv_files:
```

```
    monthly_data = pd.read_csv(file_path)
```

```
    dfs.append(monthly_data)
```

```
options_data_raw = pd.concat(dfs, ignore_index=True)
```

Data Characteristics

- Scope: Millions of option records across multiple months
- Fields: Greeks, implied volatility, bid/ask prices, volume
- Structure: Each row represents one option contract quote

Data Structure and Cleaning

Column Standardization

Before: [QUOTE_UNIXTIME], [C_DELTA], [STRIKE] After: quote_unixtime, c_delta, strike

python

Cleaning process

```
options_data_raw.columns = (
    options_data_raw.columns
    .str.strip("[]")
    .str.replace("'", "", regex=False)
    .str.replace(" ", "_", regex=False)
    .str.lower()
)
```

Data Type Conversions

- Numeric Columns: Convert Greeks and prices to float
- Date Columns: Convert to datetime format
- String Cleaning: Remove whitespace from text fields

Key Data Fields

Category	Fields	Description
Meta	symbol, expire_date, dte	Contract identification
Calls	c_delta, c_gamma, c_vega, c_theta, c_rho, c_iv	Call option Greeks

Puts	p_delta, p_gamma, p_vega, p_theta, p_rho, p_iv	Put option Greeks
Pricing	c_bid, c_ask, c_last, p_bid, p_ask, p_last	Market prices
Volume	c_volume, p_volume, c_size, p_size	Trading activity

Expiry Classification System

Classification Logic

The system categorizes options based on expiry timing:

LEAPS (Long-term)

- Criteria: Expiry ≥ 365 days
- Use Case: Long-term strategies

Monthly Expirations

- Near Month: Current month's third Friday
- Next Month: Next month's third Friday
- Far Month: Third Friday two months out
- Later Month: Any other monthly expiry

Weekly Expirations

- Criteria: Any Friday that is NOT the third Friday
- Characteristics: Shorter duration, potentially less liquid

Other Types

- End-of-Month: Last trading day of the month
- Daily: Default for all other scenarios

Implementation

```
def get_expiry_type(quote_date, expiry_date):
    days_to_expiry = (expiry_date - quote_date).days

    if days_to_expiry >= 365:
        return "LEAPS"

    is_friday = expiry_date.weekday() == 4
    third_friday = calculate_third_friday(expiry_date)

    if is_friday and expiry_date.date() == third_friday:
        return determine_monthly_type(quote_date, expiry_date)
    elif is_friday:
        return "Weekly"
    else:
        return "Daily"
```

[illegible]

2. Market Depth Filter

python

Remove illiquid contracts (0 x 0 size)

```
options_data = options_data[(options_data['c_size'] != '0 x 0') &  
                             (options_data['p_size'] != '0 x 0')]
```

3. Bid/Ask Validation

python

Ensure valid bid/ask prices

```
options_data = options_data[(options_data.c_bid != 0.0) &  
                             (options_data.p_bid != 0.0) &  
                             (options_data.c_ask != 0.0) &  
                             (options_data.p_ask != 0.0)]
```

4. Implied Volatility Filter

python

Remove contracts without a valid IV

```
options_data = options_data.dropna(subset=["c_iv", "p_iv"])
```

Strike Selection Criteria

- Strike Distance: Focus on strikes within reasonable proximity to the underlying
- Moneyness: ATM, OTM selections based on strategy requirements
- Liquidity: Prefer strikes with active trading volume

Event-Based Backtesting Implementation

Core Concept

Event-based backtesting triggers actions based on specific market events rather than fixed time intervals, providing:

- More realistic representation of trading
- Efficient processing (focuses on significant events)
- Dynamic response to market conditions

Key Components

1. Trading Day Classification

python

```
def assign_trading_day(date, expire_date):  
    if date in first_trading_days.values:  
        return 'first'  
    elif date in day_before_expiry_trading_days.values:  
        return 'day_before_expiry'  
    else:  
        return 'regular'
```

2. Strategy Setup Function

python

```
def setup_longcall(analysis_date, options_chain_data, direction='long'):  
    # Identify ATM strike  
    atm_strike = find_atm_strike(options_chain_data)  
  
    # Create strategy structure  
    strategy = pd.DataFrame({  
        'Option Type': ['CE'],  
        'Strike Price': [atm_strike],
```



```

        'position': [1 if direction == 'long' else -1]
    })

    # Get premium
    strategy['premium'] = strategy.apply(
        lambda r: get_premium(r, options_chain_data), axis=1)

    return strategy

```

3. Mark-to-Market Tracking

python

```

def add_to_mtm(mark_to_market, option_strategy, trading_date):
    option_strategy['Date'] = trading_date
    mark_to_market = pd.concat([mark_to_market, option_strategy])
    return mark_to_market

```

Backtesting Loop Structure

python

```

for trading_date in date_range:
    if entry_condition_met():
        # Enter position
        strategy = setup_strategy(trading_date, options_data)
        current_position = 1

    elif exit_condition_met():
        # Exit position

```

```
calculate_pnl()
record_trade()
current_position = 0
```

```
# Update daily MTM
update_mark_to_market()
```

Strategy Implementation Examples

Long Call Strategy

Entry Conditions

- Timing: First trading day of each month
- Strike Selection: At-the-money (ATM) call option
- Position: Buy 1 contract (position = +1)

Exit Conditions

- Primary: Day before expiry
- Alternative: Stop-loss (-20%) or Take-profit (+40%)

Implementation

python

```
def setup_longcall(analysis_date, options_chain_data, direction='long'):
    atm_strike_price = options_chain_data[
        options_chain_data.strike_distance_pct ==
        options_chain_data.strike_distance_pct.min()
    ].strike.values[0]

    long_call_data = pd.DataFrame({
```

```

'Option Type': ['CE'],
'Strike Price': [atm_strike_price],
'position': [1]
})

long_call_data['premium'] = long_call_data.apply(
    lambda r: get_premium(r, options_chain_data), axis=1)

return long_call_data

```

Long Strangle Strategy

Strategy Structure

- Call Leg: Buy OTM call (strike above ATM)
- Put Leg: Buy OTM put (strike below ATM)
- Market View: Expecting high volatility/large price movement

Implementation

python

```

def setup_long_strangle(analysis_date, options_chain_input, direction='long'):
    # Find ATM and adjacent strikes
    atm_position = find_atm_position(options_chain_input)
    call_strike = options_chain_input.iloc[atm_position - 1].strike # OTM call
    put_strike = options_chain_input.iloc[atm_position + 1].strike # OTM put

    strangle_data = pd.DataFrame({
        'Option Type': ['CE', 'PE'],
        'Strike Price': [call_strike, put_strike],

```

```
'position': [1, 1]
})

strangle_data['premium'] = strangle_data.apply(
    lambda r: get_premium(r, options_chain_input), axis=1)

return strangle_data
```

Trading Costs and Slippage

python

Approximate trading costs as a percentage of turnover

trading_cost_pct = 0.01 *# 1% of turnover*

trades['pnl_post_costs'] = trades['pnl'] - trades['turnover'] * trading_cost_pct

Performance Metrics and Analytics

Trade-Level Analytics

Basic Metrics

- Total PnL: Sum of all trade profits/losses
- Total Trades: Number of completed round trips
- Win Rate: Percentage of profitable trades
- Average Profit per Winner: Mean gain from profitable trades
- Average Loss per Loser: Mean loss from unprofitable trades

Advanced Metrics

python

Profit Factor

profit_factor = (win_rate * avg_profit_winner) / (loss_rate * avg_loss_loser)

Average Holding Time

```
holding_time = (trades['exit_date'] - trades['entry_date']).dt.days.mean()
```

Strategy-Level Analytics

Risk-Adjusted Returns

python

CAGR calculation

```
total_return = final_equity / initial_capital
```

```
trading_days = len(data)
```

```
cagr = (total_return ** (252 / trading_days) - 1) * 100
```

Annualized Volatility

```
annualized_vol = daily_returns.std() * np.sqrt(252) * 100
```

Sharpe Ratio

```
risk_free_rate = 0.02 / 252
```

```
excess_returns = daily_returns - risk_free_rate
```

```
sharpe_ratio = np.sqrt(252) * excess_returns.mean() / daily_returns.std()
```

Drawdown Analysis

python

Calculate maximum drawdown

```
cumulative_returns = (1 + daily_returns).cumprod()
```

```
running_max = cumulative_returns.cummax()
```

```
drawdown = (cumulative_returns - running_max) / running_max
```

```
max_drawdown = drawdown.min()
```

Margin and Capital Requirements

For Long Positions

python

```
capital_required = premium * lot_size
```

For Short Positions

python

SPAN + Exposure + Buffer

```
notional_value = strike_price * lot_size
```

```
span_margin = notional_value * 0.10
```

```
exposure_margin = notional_value * 0.06
```

```
buffer = notional_value * 0.03
```

```
total_margin = span_margin + exposure_margin + buffer
```

Risk Management and Optimisation

Systematic Risk Management

Stop-Loss Implementation

python

```
stoploss_pct = -0.20 # 20% stop loss
```

```
if current_return <= stoploss_pct:
```

```
    exit_position("Stoploss")
```

Take-Profit Implementation

python

```
takeprofit_pct = 0.40 # 40% take profit
if current_return >= takeprofit_pct:
    exit_position("Take Profit")
```

Position Sizing

python

Fixed lot size approach

```
lot_size = 50
```

Risk-based sizing (alternative)

```
risk_per_trade = portfolio_value * 0.02
```

```
position_size = risk_per_trade / option_premium
```

Strategy Optimization Approaches

Entry/Exit Timing

- Market Condition Filters: VIX levels, trend indicators
- Volatility-Based Entry: Enter when IV is historically cheap/expensive
- Time-Based Exits: Exit at specific DTE rather than expiry

Advanced Features

python

Volatility filter example

```
if current_iv < historical_iv_percentile(20):
```

```
    # Enter when IV is cheap
```

```
    enter_position()
```

Performance Visualization

Key Charts

1. Cumulative Returns vs Underlying: Compare strategy performance to market
2. Entry/Exit Points: Visualize trade timing on price chart
3. Monthly Return Heatmap: Identify seasonal patterns
4. Drawdown Analysis: Track worst-case scenarios

Sample Visualization Code

python

Dual-axis plot for returns vs underlying

```
fig, ax1 = plt.subplots(figsize=(12, 6))  
ax1.plot(dates, underlying_price, color='blue', label='Underlying')  
ax2 = ax1.twinx()  
ax2.plot(dates, cumulative_returns, color='red', label='Strategy Returns')  
ax1.scatter(entry_dates, entry_prices, color='green', marker='^', label='Entry')  
ax1.scatter(exit_dates, exit_prices, color='black', marker='v', label='Exit')
```

Key Takeaways and Best Practices

Framework Design Principles

1. Modularity: Build reusable functions for strategy setup and analysis
2. Flexibility: Support multiple strategy types and exit conditions
3. Realistic Simulation: Include trading costs, slippage, and margin requirements
4. Comprehensive Analysis: Track both trade-level and portfolio-level metrics

Common Pitfalls to Avoid

- Look-ahead Bias: Using future information in backtests
- Survivorship Bias: Only including contracts that didn't get delisted
- Overfitting: Excessive parameter optimization on limited data
- Unrealistic Assumptions: Ignoring liquidity constraints and execution costs

Practical Implementation Tips

- Start with simple strategies before adding complexity

- Validate results with out-of-sample testing
- Consider transaction costs and market impact
- Implement proper risk management from the beginning
- Document assumptions and limitations clearly

Future Enhancements

- Multi-Asset Support: Extend beyond SPX to other underlyings
- Advanced Strategies: Iron condors, butterflies, calendar spreads
- Real-Time Adaptation: Dynamic parameter adjustment based on market conditions

This framework provides a solid foundation for systematic options strategy research and development. The modular design allows for easy extension and customisation based on specific research objectives and market insights.